ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

BÁO CÁO ĐỒ ÁN GIỮA KỲ

Môn học: Nhận dạng mẫu

NHẬN DẠNG CHỮ SỐ VIẾT TAY BẰNG CÁC PHƯƠNG PHÁP PHÂN LỚP

Thực nghiệm trên bộ dữ liệu MNIST

Gaussian Naive Bayes · Softmax Regression · MLP
Stacking với AdaBoost · CNN và tăng cường dữ liệu

Sinh viên thực hiện

TRẦN HÙNG LUYỆN MSSV: 24110201

LÊ MỸ TÀI MSSV: 24110235

TP. HỒ CHÍ MINH, THÁNG 11 NĂM 2026

Mục lục

Tóm tắt **1**

1 Giới thiệu **1**

2 Dữ liệu và tiền xử lý **2**

3 Cơ sở phương pháp và đối chiếu mã nguồn **3**

4 Thiết kế thực nghiệm **8**

5 Kết quả và phân tích bốn phương pháp **10**

6 Mở rộng: CNN, MLP và tăng cường dữ liệu **16**

7 Thảo luận và kết luận **20**

Phân công công việc và khai báo hỗ trợ **21**

Tài liệu tham khảo **22**

A Đối chiếu mã nguồn, số liệu và khả năng tái lập **23**

B Các độ đo bổ sung trên ba tập **25**

C Khảo sát chi tiết tầng meta **26**

Quy ước đọc báo cáo

Train / Validation / Test	Tập huấn luyện / tập chọn cấu hình / tập kiểm thử.
Acc., P, R, F1	Accuracy; precision, recall và F1 trung bình macro.
OOF / CE / SGD	Out-of-fold; cross-entropy; stochastic gradient descent.
Đơn vị	Độ đo phân lớp: %; chênh lệch: điểm phần trăm; thời gian: giây.

Phần chính trình bày mô hình và kết quả. Phụ lục ghi các kiểm tra tính nhất quán, những khác biệt giữa mã nguồn và kết quả đã lưu, cùng nguồn dữ liệu dùng lập bảng. Các hình thực nghiệm được lấy từ kết quả đính kèm; hình tổng hợp được dựng lại từ CSV, không huấn luyện lại mô hình.

Tóm tắt

Báo cáo khảo sát bài toán nhận dạng mười chữ số viết tay trên MNIST bằng Gaussian Naive Bayes, Softmax Regression, mạng nơ-ron nhiều lớp (MLP) và stacking với AdaBoost ở tầng cuối. Ba bộ phân lớp đầu có phần lõi cài đặt bằng NumPy; quy trình stacking tự tổ chức việc chọn mô hình nền và tạo dự đoán out-of-fold (OOF), còn các estimator dùng scikit-learn. Theo kết quả đã lưu, accuracy trên 10 000 ảnh Test lần lượt đạt 81,49%, 92,53%, 97,63% và 98,54%. Stacking kết hợp SVM, KNN và MLP (256,128) bằng OOF năm fold; so với soft voting, mô hình sửa 20 lỗi nhưng tạo thêm 18 lỗi, chỉ tăng ròng hai ảnh đúng. Thí nghiệm mở rộng dùng CNN và MLP có gần bằng số tham số, cùng ngân sách 15 epoch. CNN không augmentation đạt 98,57% trên Test sạch. Tổ hợp xoay, dịch và nhiễu làm giảm accuracy sạch nhưng tăng độ bền đối với các phép xoay và dịch được kiểm tra. Các độ đo được đối chiếu với dự đoán hoặc confusion matrix đã lưu; những khác biệt phiên bản giữa notebook, mã nguồn và metadata được ghi riêng, nên chưa khẳng định tái lập toàn bộ lượt chạy.

Từ khóa: MNIST; Gaussian Naive Bayes; Softmax; backpropagation; stacking; AdaBoost; CNN.

1 Giới thiệu

1.1 Bài toán và mục tiêu

Cho ảnh xám $I \in \{0, \dots, 255\}^{28\times 28}$, mục tiêu là dự đoán nhãn $y \in \{0, \dots, 9\}$. Cùng một chữ số có thể khác nhau về nét viết, độ nghiêng, độ dày và vị trí; một số kiểu viết giữa hai lớp cũng có thể gần giống nhau. Đồ án dùng bài toán này để so sánh mô hình xác suất, mô hình tuyến tính, mạng phi tuyến và phương pháp kết hợp mô hình, theo yêu cầu của đề bài [1].

Ba mục tiêu thực nghiệm là: kiểm tra sự tương ứng giữa công thức và phần cài đặt; đánh giá tác động của các siêu tham số trên Validation; và phân tích chất lượng dự đoán cùng chi phí, lỗi theo lớp và khả năng giải thích. Kết quả Test được dùng để báo cáo sau khi chốt cấu hình theo quy trình mô tả trong notebook, không làm tiêu chí chọn mô hình.

1.2 Phạm vi thực hiện

Gaussian NB ước lượng tham số phân phối theo lớp; Softmax tối ưu cross-entropy với mini-batch SGD; MLP cài đặt feedforward và backpropagation. Với ensemble, sáu ứng viên thư viện được khảo sát trước khi chọn một bộ ba và học AdaBoost trên 30 đặc trưng xác suất OOF. CNN và augmentation là phần mở rộng, không thay thế việc đánh giá bốn phương pháp chính.

Phạm vi tự cài đặt cần trình bày đúng. Gaussian NB, Softmax và MLP có lõi NumPy. Các cây quyết định, mô hình nền và thuật toán AdaBoost của ensemble dùng thư viện. Vì đề bài yêu cầu tự cài phần tính toán lõi, không thể xem việc tự tổ chức stacking là bằng chứng đã tự cài đặt AdaBoost.

Báo cáo sử dụng các kết quả đã có, không tạo số liệu huấn luyện mới. Công thức và mã giả bám theo mã nguồn đính kèm; riêng các khác biệt ảnh hưởng khả năng tái lập được nêu ở Phụ lục A. Các tên và MSSV trên bìa được giữ theo bản báo cáo gốc; phân công và tỷ lệ đóng góp cần thành viên xác nhận theo công việc thực tế.

2 Dữ liệu và tiền xử lý

2.1 Cách nạp và chia dữ liệu

MNIST được đọc từ bốn tệp IDX nén gzip. Hàm đọc kiểm tra mã định dạng 2051 cho ảnh và 2049 cho nhãn; ảnh có kích thước 28×28. Loader trả về bốn mảng: ảnh/nhãn của 60 000 mẫu Train gốc và ảnh/nhãn của 10 000 mẫu Test gốc. Việc tách Validation được thực hiện trong notebook theo giao thức ở Bảng 1.

Bảng 1. Phân chia dữ liệu và vai trò trong thực nghiệm

Tập	Số ảnh	Vai trò
Train	50 000	Học tham số; tạo OOF. Với mô hình nền, chia tiếp 40 000/10 000 để tuning nội bộ.
Validation	10 000	Chọn cấu hình, checkpoint, bộ ba và trọng số voting.
Test	10 000	Đánh giá cuối trên tập kiểm thử gốc.

Phép chia dùng `train_test_split(test_size=10000, stratify=y, random_state=42)`. Kiểm tra nhãn đã lưu cho thấy Gaussian, Softmax và stacking dùng cùng thứ tự nhãn Train/Validation; các chỉ số chia tập lưu ở stacking và thí nghiệm CNN augmentation trùng nhau. Đây là bằng chứng về sự nhất quán của các artifact, không thay thế việc chạy lại toàn bộ notebook.

2.2 Biểu diễn đầu vào

Mỗi ảnh được đổi sang `float32`, chia cho 255 và làm phẳng:

$$X_{i,j} = \frac{I_{i,j}}{255}, \qquad X \in [0,1]^{N \times 784}. \tag{1}$$

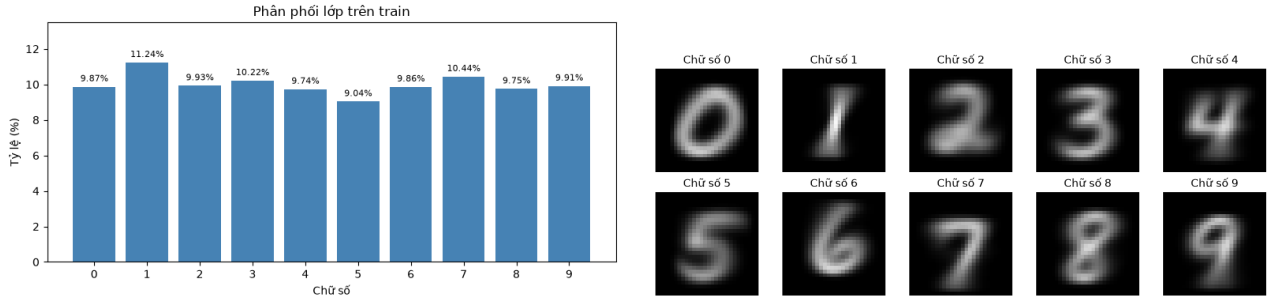
Đây là phép đổi thang cố định; mã nguồn không trừ mean hoặc chia standard deviation. Không có bước PCA, chuẩn hóa z-score hay nhân bản dữ liệu trong pipeline chính. CNN nhận cùng cường độ pixel, chỉ reshape thành $N \times 1 \times 28 \times 28$. Nhãn giữ dạng số nguyên, không lưu thành vector one-hot.

Gaussian dùng `float64`. Notebook MLP chuyển đầu vào sang `float64`, nhưng `fit` của lớp MLP hiện có lại ép về `float32`; mô hình MLP đã lưu chứa tham số `float64`. Vì vậy, không thể gộp hai thông tin này thành khẳng định "cả hai cài đặt MLP đều huấn luyện float64"; chi tiết được ghi ở Phụ lục A.

2.3 Khám phá dữ liệu

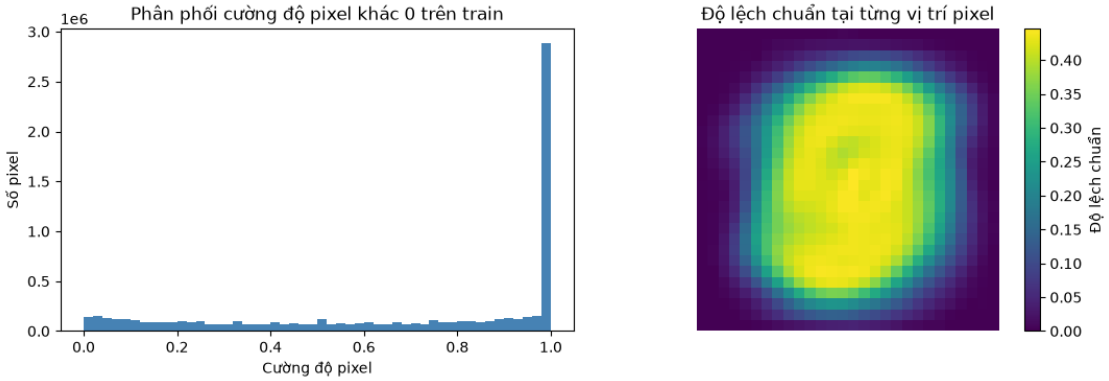
Hình 1 trích trực tiếp output đã lưu của notebook EDA, trên 50 000 ảnh Train. Lớp 1 có 5 618 ảnh (11,236%), nhiều nhất; lớp 5 có 4 518 ảnh (9,036%), ít nhất. Tỷ lệ chênh lệch không lớn nhưng các lớp chưa hoàn toàn cân bằng, nên báo cáo thêm các độ đo macro. Trên Train, 80,86% pixel bằng 0; có 72/784 vị trí luôn bằng 0. Tỷ lệ pixel đúng bằng 1 là 0,66%. Các giá trị này là thống kê cường độ chính xác, không phải phân đoạn nền theo một ngưỡng tùy chọn.

Ảnh trung bình thể hiện vùng nét thường xuất hiện trong từng lớp nhưng làm mờ các kiểu viết riêng. Pixel ở vùng biên có thể có phương sai bằng hoặc gần 0, là lý do cần khảo sát lượng làm trơn của Gaussian NB. Mối liên hệ không gian giữa các pixel cũng gợi ý hạn chế của giả định độc lập có điều kiện. Những quan sát EDA này chỉ định hướng phân tích; mức cải thiện dự đoán phải được xác định bằng thí nghiệm phân lớp.



(a) Phân phối lớp trên Train. **(b)** Ảnh trung bình theo nhãn.

Hình 1. EDA từ output notebook: phân phối lớp và ảnh trung bình của 50 000 ảnh Train.



Hình 2. Phân phối các pixel khác 0 và độ lệch chuẩn tại từng vị trí, lấy từ output EDA trên Train.

3 Cơ sở phương pháp và đối chiếu mã nguồn

Mỗi hàng của X là một ảnh. Ký hiệu n là số mẫu Train, m là số mẫu trong mini-batch, $d = 784$, $K = 10$; $\mathbf{1}[\cdot]$ là hàm chỉ báo và $\mathbf{1}_m$ là vector cột gồm m số 1. Các công thức dưới đây dùng cùng chiều ma trận và quy ước chuẩn hóa gradient như phần cài đặt.

3.1 Gaussian Naive Bayes

Gaussian NB giả định các pixel độc lập có điều kiện theo lớp và mỗi pixel tuân theo phân phối Gaussian [2]. Với $I_k = \{i : y_i = k\}$ và $n_k = |I_k|$, phương thức `fit` tính

$$\pi_k = \frac{n_k}{n}, \quad \mu_{kj} = \frac{1}{n_k} \sum_{i \in I_k} x_{ij}, \quad v_{kj} = \frac{1}{n_k} \sum_{i \in I_k} (x_{ij} - \mu_{kj})^2 + \varepsilon. \tag{2}$$

Mẫu số là n_k, tương ứng `np.var(ddof=0)`. Tham số `var_smoothing` của lớp tự viết chính là lượng cộng tuyệt đối ε, không phải hệ số nhân phương sai. Điểm số và dự đoán trong `_log_joint`, `predict` là

$$s_k(x) = \log \pi_k - \frac{1}{2} \sum_{j=1}^{d} \left[\log(2\pi v_{kj}) + \frac{(x_j - \mu_{kj})^2}{v_{kj}} \right], \qquad \hat{y} = \arg\max_k s_k(x). \tag{3}$$

Chỉ cần so sánh các log-joint vì mẫu số Bayes giống nhau giữa các lớp. Phép cộng log tránh nhân rất nhiều mật độ nhỏ. Epsilon dương giúp xử lý các pixel có phương sai 0.

Mã giả 1. Huấn luyện và dự đoán Gaussian NB

Vào: $X \in \mathbb{R}^{n\times d}$, nhãn y, $\varepsilon > 0$.
Lấy các lớp theo thứ tự tăng dần.
Với mỗi lớp k: lấy $X_k = X[y = k]$.
  Tính π_k, μ_k, v_k theo công thức (2).
Lưu prior, mean, variance của mười lớp.
Dự đoán: tính mười điểm $s_k(x)$ theo (3).
Trả về nhãn tại vị trí có điểm lớn nhất.

Để đối chiếu scikit-learn, cần quy đổi cùng lượng cộng vào phương sai [4]:

$$\texttt{var_smoothing}_{\text{sklearn}} = \frac{\varepsilon}{\max_j \mathrm{Var}_{\text{Train}}(X_{:,j})}. \tag{4}$$

So sánh với cùng tên tham số nhưng khác quy ước sẽ tạo hai mô hình khác nhau. Mô hình lưu hai ma trận 10×784 và mười prior, tổng cộng 15 690 giá trị tham số (chưa trừ các ràng buộc giữa prior).

3.2 Softmax Regression

Lan truyền xuôi và hàm mục tiêu

Softmax là logistic regression đa lớp với $W \in \mathbb{R}^{784\times 10}$ và $b \in \mathbb{R}^{10}$ [2, 5]. Trong mini-batch:

$$Z = X_B W + \mathbf{1}_m b^\top, \qquad a_i = \max_k Z_{ik}, \tag{5}$$

$$P_{ik} = \frac{\exp(Z_{ik} - a_i)}{\sum_{r=0}^{9} \exp(Z_{ir} - a_i)}, \qquad \hat{y}_i = \arg\max_k P_{ik}. \tag{6}$$

Phép `X @ W + b` trong NumPy broadcast b lên mọi hàng, đúng với $\mathbf{1}_m b^\top$. Trừ a_i không đổi xác suất và làm ổn định hàm mũ. Gradient trong `_grad` tương ứng hàm mục tiêu

$$\mathcal{L}_B(W, b) = -\frac{1}{m}\sum_{i=1}^{m} \log P_{i,y_i} + \frac{\lambda}{2}\|W\|_F^2. \tag{7}$$

Không phạt bias. Đặt $E_{ik} = P_{ik} - \mathbf{1}[k = y_i]$, các gradient là

$$g_W = \frac{X_B^\top E}{m} + \lambda W, \quad g_b = \frac{E^\top \mathbf{1}_m}{m}, \qquad W \leftarrow W - \eta g_W, \quad b \leftarrow b - \eta g_b. \tag{8}$$

Mã tạo E bằng cách trừ 1 tại từng ô (i, y_i); không cần tạo ma trận one-hot. Khởi tạo $W = 0$, $b = 0$ cho $P_{ik} = 0, 1$ ở thời điểm ban đầu. Softmax có 7 850 tham số.

Loss theo dõi, checkpoint và regularization

Hàm `_loss` chỉ trả về cross-entropy, không cộng L2:

$$\mathrm{CE}(X, y) = \frac{1}{N}\sum_{i=1}^{N}\left[\log\sum_{k=0}^{9}\exp(Z_{ik} - a_i) - (Z_{i,y_i} - a_i)\right]. \tag{9}$$

Do đó, đường CE không phải toàn bộ hàm mục tiêu (7). Mỗi cấu hình chạy 100 epoch với `patience=None`. Khi CE Validation giảm quá 10^{-6} so với checkpoint đang giữ, code lưu bản sao

tham số. Kết thúc 100 epoch, code khôi phục bản sao đó; đây là chọn checkpoint, không phải dừng sớm.

Với dữ liệu không có trọng số mẫu và cùng L2, logistic regression của scikit-learn dùng chuẩn hóa tương đương $\lambda = 1/(nC)$ [5]. Bởi vậy, $n = 50\,000$, $\lambda = 2 \times 10^{-5}$ tương ứng $C = 1$. L-BFGS và SGD có lịch cập nhật, điều kiện dừng khác nhau; bằng nhau về regularization không đòi hỏi hai bộ tham số hoặc dự đoán trùng tuyệt đối.

Mã giả 2. Softmax Regression bằng mini-batch SGD

Vào: Train, Validation, η, λ, batch 256, 100 epoch, seed 42.
Khởi tạo $W = 0$, $b = 0$, bestCE $= +\infty$.
Mỗi epoch: xáo trộn Train và duyệt các mini-batch.
  Tính P theo (6); đặt $E = P$ rồi trừ 1 tại lớp thật.
  Tính g_W, g_b và cập nhật theo (8).
Ghi CE và accuracy Train/Validation tại cuối epoch.
Nếu $\text{CE}_{\text{val}} < \text{bestCE} - 10^{-6}$: cập nhật bestCE, lưu W, b, epoch.
Sau đủ 100 epoch, khôi phục checkpoint đã lưu.
Suy luận: tính P và lấy nhãn có xác suất lớn nhất.

3.3 MLP tự cài đặt bằng NumPy

Lan truyền xuôi và khởi tạo

Đặt kích thước các lớp là $(d_0, d_1, \dots, d_L)$, $d_0 = 784$, $d_L = 10$; $A^{(0)} = X_B$. Mã dùng ReLU ở lớp ẩn và softmax ở đầu ra [2]:

$$\begin{aligned} Z^{(\ell)} &= A^{(\ell-1)} W^{(\ell)} + \mathbf{1}_m (b^{(\ell)})^\top, \\ A^{(\ell)} &= \max(0, Z^{(\ell)}) \quad (\ell < L), \qquad P = \text{softmax}(Z^{(L)}). \end{aligned} \tag{10}$$

Trong file `src/models/mlp.py` hiện có, cả trọng số và bias được lấy ngẫu nhiên đều:

$$W^{(\ell)}_{ab},\ b^{(\ell)}_a \sim \mathcal{U}(-r_\ell, r_\ell), \qquad r_\ell = \sqrt{\frac{6}{d_{\ell-1} + d_\ell}}. \tag{11}$$

Đây là dạng Glorot uniform trong mã hiện tại, không phải He normal; bias cũng không được đặt bằng 0. Các mảng khởi tạo được ép `float32`. Vì artifact MLP đã lưu có dtype khác, công thức (11) mô tả phiên bản mã được cung cấp, chưa xác nhận được khởi tạo thực sự của lượt chạy đã tạo ra các số liệu MLP.

Lan truyền ngược

Với $Y_{ik} = \mathbf{1}[k = y_i]$, code tính $D^{(L)} = (P - Y)/m$ và duyệt các lớp theo chiều ngược:

$$g_{W^{(\ell)}} = (A^{(\ell-1)})^\top D^{(\ell)} + \lambda W^{(\ell)}, \qquad g_{b^{(\ell)}} = (D^{(\ell)})^\top \mathbf{1}_m, \tag{12}$$

$$D^{(\ell-1)} = \left[D^{(\ell)} (W^{(\ell)})^\top\right] \odot \mathbf{1}[A^{(\ell-1)} > 0], \qquad \ell > 1. \tag{13}$$

Sai số lớp trước được tính với trọng số *chưa cập nhật*; sau đó mới thực hiện $W^{(\ell)} \leftarrow W^{(\ell)} - \eta g_{W^{(\ell)}}$ và cập nhật bias. Đạo hàm ReLU tại 0 được chọn bằng 0. Hệ số $1/m$ đã nằm trong D, nên không chia thêm cho m ở gradient trọng số.

Hàm loss trong mã trả về

$$\mathcal{L}_{\text{code}} = -\frac{1}{N}\sum_i \log(P_{i,y_i} + 10^{-12}) + \frac{\lambda}{2}\sum_{\ell=1}^{L}\|W^{(\ell)}\|_F^2. \tag{14}$$

Backpropagation dùng gradient CE chuẩn; hằng 10^{-12} chỉ được thêm trong log khi ghi loss. Vì thế gradient không phải đạo hàm hoàn toàn chính xác của biểu thức có cộng hằng số, dù sai khác không đáng kể khi xác suất lớp thật lớn hơn nhiều 10^{-12}. Trong thí nghiệm MLP đã lưu, $\lambda = 0$, chạy 20 epoch và dùng tham số cuối cùng; không có early stopping hoặc khôi phục checkpoint.

Mã giả 3. Feedforward và backpropagation của MLP

Vào: Train, kích thước các lớp, η, λ, batch 256, 20 epoch, seed 42.
Khởi tạo các ma trận và bias theo (11) của mã hiện tại.
Mỗi epoch: xáo trộn Train, duyệt từng mini-batch gồm m ảnh.
  Lan truyền xuôi theo (10), lưu các activation.
  Đặt $D = P$; trừ 1 tại lớp thật, rồi chia D cho m.
  Với $\ell = L, L-1, \dots, 1$:
    Tính g_W, g_b theo (12).
    Nếu $\ell > 1$, tính $D_{\text{trước}}$ theo (13) trước khi đổi W.
    Cập nhật trọng số, bias bằng SGD; chuyển sang $D_{\text{trước}}$.
Ghi loss Train cuối epoch. Trả về tham số sau epoch cuối.

Kiến trúc được chọn theo bảng khảo sát là 784–256–128–64–10, có $(784+1)256 + (256+1)128 + (128+1)64 + (64+1)10 = 242\,762$ tham số. MLP NumPy này khác với MLP (256,128) dùng Adam trong ensemble và MLP260 dùng đối chiếu CNN; ba mô hình không dùng chung trọng số.

3.4 Lựa chọn mô hình nền và voting

Sáu ứng viên gồm SVM RBF, logistic regression, Random Forest, Extra Trees, KNN và MLP (256,128). SVM dùng kernel $k(x, x') = \exp(-\gamma\|x - x'\|^2)$; KNN dùng ba láng giềng với trọng số theo khoảng cách. MLP nền là estimator scikit-learn dùng Adam, không phải lớp NumPy ở trên. Các mô hình trả về xác suất theo thứ tự nhãn 0–9.

Gọi $E_j = \{i : \hat{y}_i^{(j)} \neq y_i\}$ là tập lỗi Validation của mô hình j. Chương trình tính

$$\text{common}(S) = \left|\bigcap_{j\in S} E_j\right|, \qquad \text{overlap}(S) = \frac{\text{common}(S)}{\min_{j\in S}|E_j|}, \tag{15}$$

$$DF(a,b) = \frac{1}{N_{\text{val}}}\sum_i \mathbf{1}[i \in E_a]\mathbf{1}[i \in E_b], \qquad \text{oracle}(S) = 1 - \frac{\text{common}(S)}{N_{\text{val}}}. \tag{16}$$

Trùng lỗi nghĩa là cùng sai trên một ảnh, không yêu cầu cùng dự đoán một nhãn sai. Mean double-fault là trung bình DF của ba cặp trong một bộ ba. Oracle giả định biết nhãn thật để chọn một thành viên đúng, nên chỉ là chỉ số chẩn đoán; nó không phải mô hình có thể triển khai và không phải cận trên tổng quát của một meta-learner.

Soft voting và weighted soft voting kết hợp ba vector xác suất:

$$\bar{p}_k(x) = \sum_{j=1}^{3} w_j p_{jk}(x), \qquad w_j \geq 0, \quad \sum_j w_j = 1, \qquad \hat{y} = \arg\max_k \bar{p}_k(x). \tag{17}$$

Soft voting đặt $w_j = 1/3$. Nhóm được xếp theo accuracy voting, F1 macro, mean double-fault thấp hơn và tên nhóm. Vì chất lượng voting đứng trước độ trùng lỗi, không mô tả việc chọn nhóm là "chọn ba mô hình ít trùng lỗi nhất". SVM dùng $\arg\max$(`predict_proba`) thống nhất với voting; quy tắc này có thể khác `SVC.predict` sau bước ước lượng xác suất.

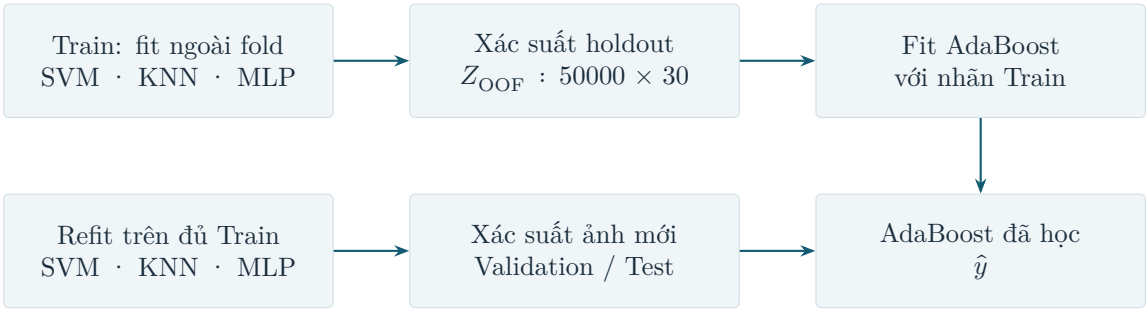
3.5 Stacking với AdaBoost ở tầng meta

Đặc trưng OOF và suy luận

Train được chia thành năm fold phân tầng $F_1, \dots, F_5$, xáo trộn với seed 42. Với mẫu $i \in F_q$, mô hình nền j được fit trên các fold còn lại, cho xác suất $p_j^{(-q)}(x_i) \in \mathbb{R}^{10}$. Ghép ba vector theo thứ tự SVM, KNN, MLP:

$$z_i = \left[p_1^{(-q)}(x_i) \,\|\, p_2^{(-q)}(x_i) \,\|\, p_3^{(-q)}(x_i)\right] \in \mathbb{R}^{30}, \qquad Z_{\text{OOF}} \in \mathbb{R}^{50000\times 30}. \tag{18}$$

AdaBoost fit trên $(Z_{\text{OOF}}, y_{\text{Train}})$. Khi dự đoán Validation/Test, các mô hình nền được refit trên đủ 50 000 ảnh; xác suất của chúng được ghép theo cùng thứ tự và đưa vào tầng meta [3, 7].



Hình 3. Hai luồng của stacking: OOF dùng học tầng meta; mô hình nền refit dùng tạo đầu vào khi suy luận.

OOF bảo đảm base model tạo đặc trưng cho một ảnh không được fit trên chính ảnh đó. Tuy nhiên, siêu tham số nền đã được chọn trước trên một phần Train, nên đây không phải nested cross-validation của toàn bộ quá trình chọn mô hình. Điểm Train sau refit là in-sample, không phải accuracy OOF của hệ thống.

AdaBoost đa lớp SAMME

Tầng meta dùng `AdaBoostClassifier` với cây quyết định có độ sâu tối đa 2, $T = 100$, learning rate $\eta_A = 0,2$. Với trọng số mẫu chuẩn hóa $u_i^{(1)} = 1/n$, mỗi cây h_t được fit trên 30 đặc trưng. Khi $0 < e_t < 1 - 1/K$, SAMME cập nhật [6]:

$$e_t = \sum_i u_i^{(t)} \mathbf{1}[h_t(z_i) \neq y_i], \tag{19}$$

$$\alpha_t = \eta_A \left[\log \frac{1 - e_t}{e_t} + \log(K - 1)\right], \tag{20}$$

$$u_i^{(t+1)} = \frac{u_i^{(t)} \exp\{\alpha_t \mathbf{1}[h_t(z_i) \neq y_i]\}}{\sum_r u_r^{(t)} \exp\{\alpha_t \mathbf{1}[h_t(z_r) \neq y_r]\}}. \tag{21}$$

Dự đoán tương đương chọn lớp có tổng phiếu trọng số lớn nhất:

$$\hat{y} = \arg\max_k \sum_{t=1}^{T} \alpha_t \mathbf{1}[h_t(z) = k]. \tag{22}$$

Thư viện xử lý riêng trường hợp cây không mắc lỗi hoặc không tốt hơn đoán ngẫu nhiên; lượt chạy cuối lưu đủ 100 cây. Đây là boosting trên xác suất OOF, không phải AdaBoost trực tiếp trên 784 pixel. Các công thức mô tả thuật toán estimator sử dụng; không đồng nghĩa nhóm tự viết lõi SAMME.

Mã giả 4. Pipeline stacking được tổ chức trong notebook 06

Nạp ba estimator và cấu hình đã chọn từ notebook 05; giữ Test ngoài quá trình chọn.
Tạo năm fold phân tầng; khởi tạo Z_{OOF} kích thước 50000×30.
Với mỗi base model j và mỗi fold q:
    Clone estimator; fit trên bốn fold còn lại; dự đoán xác suất fold q.
    Ghi đúng hàng holdout, đúng mười cột của mô hình j.
Refit các base model trên đủ Train; tạo Z_{val}.
Fit các cấu hình AdaBoost trên OOF, chọn theo accuracy rồi F1 Validation.
Fit lại cấu hình meta đã chọn; chốt mô hình đơn tham chiếu và trọng số voting.
Với ảnh mới: ghép xác suất base model đã refit, rồi gọi `meta.predict`.

4 Thiết kế thực nghiệm

4.1 Giao thức lựa chọn cấu hình

Gaussian, Softmax và MLP được fit trên 50 000 ảnh, chọn siêu tham số bằng Validation. Mô hình nền ensemble có thêm bước tuning trên phép chia 40 000/10 000 nằm trong Train, sau đó refit trên đủ Train. Validation chung được dùng để chọn bộ ba, trọng số voting và tham số tầng meta. Vì được tái sử dụng cho nhiều lựa chọn, điểm Validation có thể lạc quan; không xem đây là một đánh giá độc lập cuối cùng.

Bảng 2. Không gian khảo sát và quy tắc lựa chọn trong các notebook

Thành phần	Cấu hình khảo sát	Chọn trên Validation
Gaussian NB	$\varepsilon = 10^{-11}, 10^{-10}, \ldots, 10^{0}$	F1 macro cao nhất.
Softmax	$\eta \in \{0,001; 0,01; 0,1; 0,5; 1\}$; sau đó $\lambda \in \{0; 2\times10^{-5}; 10^{-4}; 10^{-3}; 10^{-2}\}$	F1 macro; CE thấp hơn phá hòa. Checkpoint theo CE.
MLP NumPy	6 cấu hình: thay learning rate hoặc kiến trúc; 20 epoch, L2 bằng 0	F1 macro, accuracy, ít tham số, rồi model ID.
Tổ hợp nền	6 ứng viên tạo 15 cặp và 20 bộ ba	Accuracy soft voting, F1, mean double-fault, rồi tên.
Voting weights	7 bộ: đều; lần lượt tăng một thành viên lên 2 hoặc 3	Accuracy, F1; hòa thì giữ bộ xuất hiện trước.
AdaBoost meta	Thay η_A, depth hoặc số cây quanh baseline	Accuracy, F1; rồi depth, số cây, learning rate nhỏ hơn.

Khảo sát Softmax là tuần tự: chọn learning rate tại $\lambda = 1/50\,000$, rồi chọn L2 tại learning rate đã chọn. MLP thay từng yếu tố quanh baseline (256,128), $\eta = 0,08$. AdaBoost có baseline $(\eta_A, \text{depth}, T) = (0,5; 2; 100)$: thử $\eta_A \in \{0,01; 0,1; 0,2; 0,5; 1\}$, depth 1–4 và các mốc số cây $\{1, 20, 50, 100, 200, 300\}$. Dãy số cây được lấy từ `staged_predict` của một lần fit 300 cây. Các khảo sát này không duyệt toàn bộ tích Descartes, nên chưa tìm tối ưu toàn cục.

Tuning nền ưu tiên accuracy, F1 macro phá hòa. Các giá trị được chọn lần lượt là $C = 10$, $C = 0,1$, min_samples_leaf $= 1$ cho cả hai mô hình cây, $k = 3$ và $\alpha = 0,01$. Không quy đổi trực tiếp α của

Bảng 3. Tuning mô hình nền trên 40 000/10 000 ảnh bên trong Train

Ứng viên	Cấu hình cố định chính	Lưới tham số
SVM	RBF; bật xác suất khi refit; $\gamma =$ `scale`	$C = 1, 3, 10$
Logistic Regression	L-BFGS, max_iter 1000	$C = 0,01; 0,1; 1$
Random Forest	200 cây, seed 42	min_samples_leaf $= 1, 2, 4$
Extra Trees	200 cây, seed 42	min_samples_leaf $= 1, 2, 4$
KNN	weights = distance	$k = 3, 5, 7$
MLP (256,128)	ReLU, Adam, batch 256, 60 epoch	$\alpha = 10^{-4}, 10^{-3}, 10^{-2}$

MLP thư viện thành λ của lớp NumPy, vì quy ước chuẩn hóa có thể khác.

4.2 Độ đo và phân tích sai khác dự đoán

Confusion matrix C có hàng là nhãn thật, cột là nhãn dự đoán. Với lớp k:

$$\begin{aligned} TP_k &= C_{kk}, & FP_k &= \sum_{i \neq k} C_{ik}, & FN_k &= \sum_{j \neq k} C_{kj}, \\ \mathrm{Acc} &= \frac{\sum_k C_{kk}}{N}, & P_k &= \frac{TP_k}{TP_k + FP_k}, & R_k &= \frac{TP_k}{TP_k + FN_k}, \\ F1_k &= \frac{2P_k R_k}{P_k + R_k}, & P_{\mathrm{macro}} &= \frac{1}{10}\sum_k P_k, & R_{\mathrm{macro}} &= \frac{1}{10}\sum_k R_k. \end{aligned} \tag{23}$$

$F1_{\mathrm{macro}} = \frac{1}{10}\sum_k F1_k$, không phải F1 tính lại từ P_{macro} và R_{macro}. Mẫu số bằng 0 được xử lý bằng 0, tương ứng `zero_division=0`. Recall của một chữ số là tỷ lệ ảnh thật thuộc chữ số đó được dự đoán đúng.

Với hai mô hình trên cùng Test, đếm số ảnh tham chiếu sai nhưng mô hình mới đúng (b), và tham chiếu đúng nhưng mô hình mới sai (c). Chênh lệch accuracy bằng $100(b - c)/N$ điểm phần trăm; trên 10 000 ảnh, một ảnh tương ứng 0,01 điểm. Báo cáo tính bổ sung kiểm định cặp nhị thức chính xác hai phía (McNemar exact), $B \sim \mathrm{Binomial}(b + c, 1/2)$, từ số đếm đã lưu. Kiểm định này xét cặp mô hình cố định trên các ảnh Test, không đo biến động khi đổi seed huấn luyện.

4.3 Kiểm chứng cài đặt và phạm vi đo thời gian

Gaussian được đối chiếu prior, mean và variance sau khi quy đổi epsilon. Softmax có kiểm tra gradient bằng sai phân trung tâm trên dữ liệu nhỏ ở float64. Đối với MLP, báo cáo bổ sung kiểm tra backpropagation hiện tại trên mạng nhỏ 4–5–4–3, đổi tham số sang float64 để hạn chế sai số số học; kết quả này được tách khỏi thí nghiệm MNIST đã lưu. Đối chiếu MLP sklearn dùng cùng kiến trúc, SGD không momentum, batch 256, learning rate và 20 epoch; khởi tạo và shuffle có thể khác nhau.

Thời gian lấy từ `metrics.csv` của từng mô hình. Thời gian fit lặp lại ở ba dòng split là cùng một lần fit, không cộng ba lần. Stacking gồm tạo OOF, refit nền và fit meta cuối; chưa gồm tuning nền và khảo sát meta. CNN dùng bộ kết quả augmentation 15 epoch để bảo đảm cùng ngân sách khi so sánh hai chế độ. Không trộn với bộ CNN 50 epoch ở thư mục khác.

Các số đo chạy một lần, thiếu thông tin CPU/BLAS và mức tải máy đầy đủ, nên chỉ phản ánh lượt đo đã ghi nhận. Metadata ghi scikit-learn 1.9.1; thí nghiệm CNN ghi PyTorch 2.15.0+cpu, thiết bị CPU. Các chuỗi phiên bản được trích từ artifact, không phải môi trường biên tập báo cáo.

5 Kết quả và phân tích bốn phương pháp

5.1 Kết quả tổng hợp trên Train, Validation và Test

Bảng 4. Bốn phương pháp chính trên ba tập. Tất cả độ đo tính bằng %; P, R và F1 là macro.

Phương pháp	Tập	Acc.	P	R	F1
Gaussian NB	Train	80,288	81,895	79,900	80,099
Gaussian NB	Validation	79,510	81,344	79,117	79,339
Gaussian NB	Test	81,490	83,051	81,131	81,321
Softmax NumPy	Train	93,330	93,253	93,232	93,232
Softmax NumPy	Validation	92,280	92,196	92,187	92,180
Softmax NumPy	Test	92,530	92,440	92,409	92,410
MLP NumPy	Train	99,494	99,497	99,490	99,493
MLP NumPy	Validation	97,560	97,551	97,544	97,545
MLP NumPy	Test	97,630	97,633	97,596	97,612
Stacking AdaBoost	Train	99,998	99,998	99,998	99,998
Stacking AdaBoost	Validation	98,530	98,523	98,526	98,524
Stacking AdaBoost	Test	98,540	98,540	98,534	98,536

Nguồn: evidence/{gaussian,softmax,mlp,stacking_adaboost}/metrics.csv.

Bảng 4 cho thấy Gaussian NB đạt 81,49% Test, Softmax đạt 92,53%, MLP NumPy đạt 97,63%, còn stacking đạt 98,54%. Số lỗi tương ứng là 1 851, 747, 237 và 146 ảnh. Các số này được kiểm tra lại từ dự đoán hoặc confusion matrix, không lấy từ bảng xếp hạng tổng hợp cũ. So với Gaussian, Softmax tăng 11,04 điểm phần trăm; MLP tăng 5,10 điểm so với Softmax. Đây là so sánh các cấu hình đã khảo sát trên cùng giao thức, không cô lập riêng tác động của một yếu tố như số tham số hay optimizer.

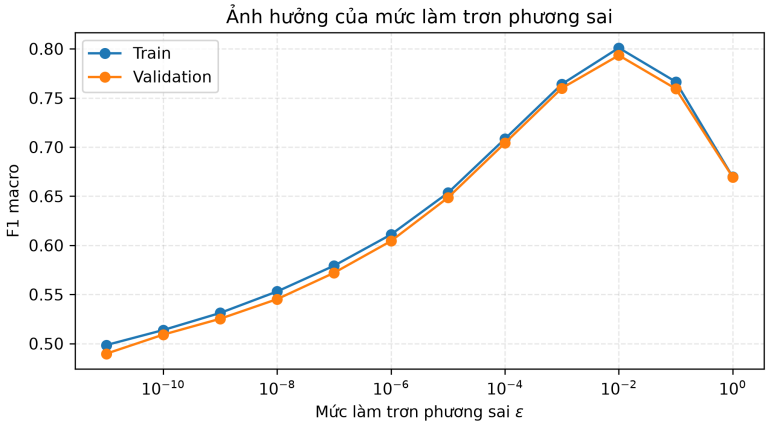
Bảng 5. Đối chiếu bản NumPy và thư viện trên Test. Thời gian theo từng lượt đo, đơn vị giây.

Cài đặt	Acc. (%)	F1 (%)	Fit	Predict 10 000 ảnh
Gaussian NB	81,490	81,321	0,338	1,466
Gaussian sklearn	81,490	81,321	0,546	0,763
Softmax NumPy	92,530	92,410	11,151	0,009
Softmax sklearn	92,410	92,297	431,213	0,016
MLP NumPy	97,630	97,612	71,609	0,069
MLP sklearn	97,070	97,046	86,699	0,126

Gaussian có các độ đo trùng bản sklearn ở cả ba tập, với dự đoán lưu trùng nhau. Softmax NumPy cao hơn bản L-BFGS 0,12 điểm trên Test; MLP NumPy cao hơn bản sklearn 0,56 điểm. Chênh lệch này không chứng minh cài tay luôn tốt hơn thư viện: budget, checkpoint, khởi tạo, precision số học và mức hội tụ khác nhau. Notebook MLP giữ cảnh báo chưa hội tụ sau 20 epoch.

5.2 Gaussian NB: tác động của làm trơn phương sai

F1 Validation tăng từ 48,972% ở $\varepsilon = 10^{-11}$ lên 79,339% ở $\varepsilon = 10^{-2}$, rồi giảm còn 75,938% ở 10^{-1} và 66,943% ở 1. Giá trị chọn là $\varepsilon = 0,01$, nằm bên trong lưới khảo sát. So với baseline 10^{-9}, mức chọn sửa 2 540 lỗi Validation nhưng tạo 237 lỗi mới, tăng ròng 2 303 ảnh đúng, đúng bằng mức tăng accuracy 23,03 điểm.



Hình 4. F1 macro Train và Validation theo epsilon cộng trực tiếp vào phương sai.

Epsilon nhỏ có thể làm pixel gần hằng số chi phối điểm log vì xuất hiện trong mẫu số phương sai. Tăng epsilon hạn chế ảnh hưởng đó; tăng quá lớn làm mất khác biệt phương sai giữa các lớp. Đây là diễn giải phù hợp với công thức và xu hướng khảo sát, chưa phải thí nghiệm cô lập đóng góp của từng pixel. Gaussian vẫn có accuracy Train chỉ 80,288%, gợi ý giới hạn mô hình với biểu diễn pixel và giả định độc lập, thay vì chỉ quy lỗi cho overfitting.

Tại $\varepsilon = 10^{-9}$, kiểm tra tham số lưu báo sai khác lớn nhất bằng 0 cho prior, mean và variance giữa hai bản; độ đo Validation cũng trùng. Ở cấu hình cuối $0,01$, các vector dự đoán đã lưu trùng nhau trên cả ba tập. Hai mức epsilon này là hai phép đối chiếu khác nhau, không được gộp thành cùng một thí nghiệm kiểm tra tham số.

5.3 Softmax: learning rate, L2 và giới hạn mô hình tuyến tính

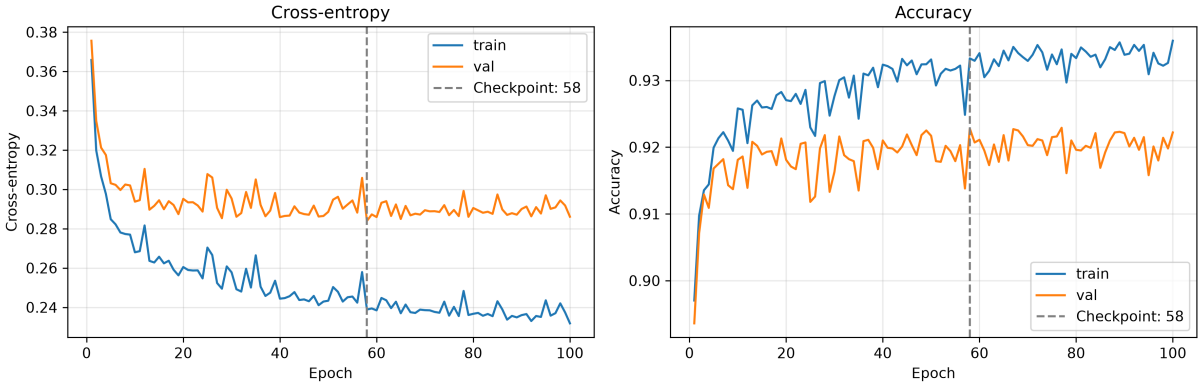
Bảng 6. Khảo sát learning rate tại $\lambda = 2 \times 10^{-5}$; kết quả tại checkpoint đã khôi phục.

η	Acc. Validation (%)	F1 Validation (%)	CE Validation	Epoch chọn
0,001	87,600	87,418	0,4974	100
0,01	90,870	90,775	0,3266	100
0,1	92,110	92,016	0,2853	96
0,5	92,280	92,180	0,2839	58
1	92,270	92,165	0,2895	58

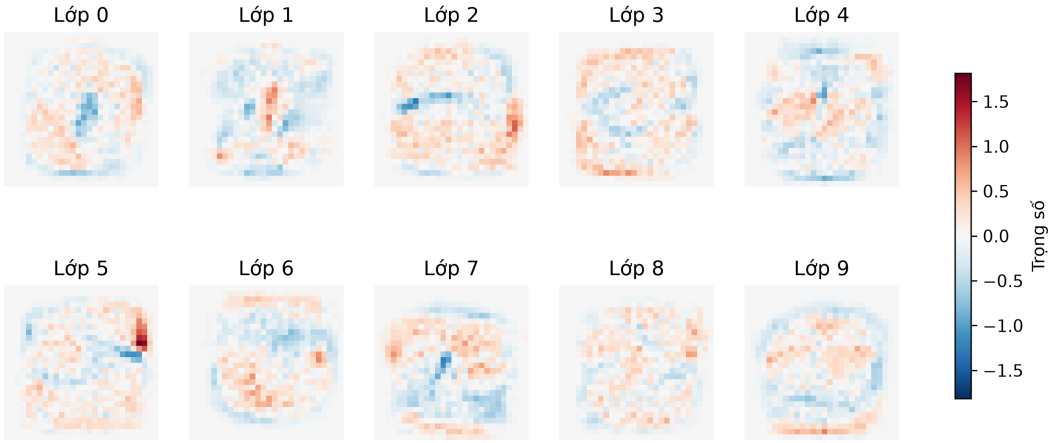
Learning rate 0,001 chưa học đủ nhanh trong ngân sách cố định, đạt F1 Validation 87,418%. Giá trị 0,5 cho F1 cao nhất trong khảo sát, 92,180%; tăng lên 1 không cải thiện. Sau khảo sát L2, cấu hình chọn vẫn là $\eta = 0,5$, $\lambda = 2 \times 10^{-5}$, batch 256. Chương trình chạy đủ 100 epoch nhưng khôi phục checkpoint epoch 58.

Ở $\lambda = 10^{-4}$, CE Validation 0,283018 thấp hơn 0,283946 của cấu hình chọn, nhưng F1 thấp hơn (92,162% so với 92,180%). Vì tiêu chí chọn cấu hình là F1 trước CE, giữ 2×10^{-5} là đúng với quy tắc đã nêu. Khi $\lambda = 10^{-2}$, F1 Validation giảm còn 89,575%; không thể kết luận phạt mạnh hơn luôn tốt hơn. Kiểm tra gradient đã lưu cho sai khác tuyệt đối lớn nhất $4,78 \times 10^{-11}$ đối với W và $2,96 \times 10^{-11}$ đối với b, phù hợp với gradient (8).

Với các đại lượng khác cố định, tăng pixel có trọng số dương làm tăng logit tương ứng; trọng số âm làm giảm logit. Xác suất còn phụ thuộc logits các lớp khác. Vì vậy, bản đồ trọng số không phải ảnh mẫu hay giải thích nhân quả. Softmax cộng tuyến tính theo vị trí pixel cố định; mô hình không có



Hình 5. CE và accuracy Softmax của toàn bộ 100 epoch; đường dọc đánh dấu checkpoint epoch 58. Các điểm sau mốc này vẫn được huấn luyện nhưng không được chọn làm tham số cuối.



Hình 6. Mười cột của ma trận W được reshape về 28×28. Màu thể hiện trọng số tác động lên logit, không phải xác suất lớp.

cơ chế tích chập hoặc học đặc trưng phân cấp.

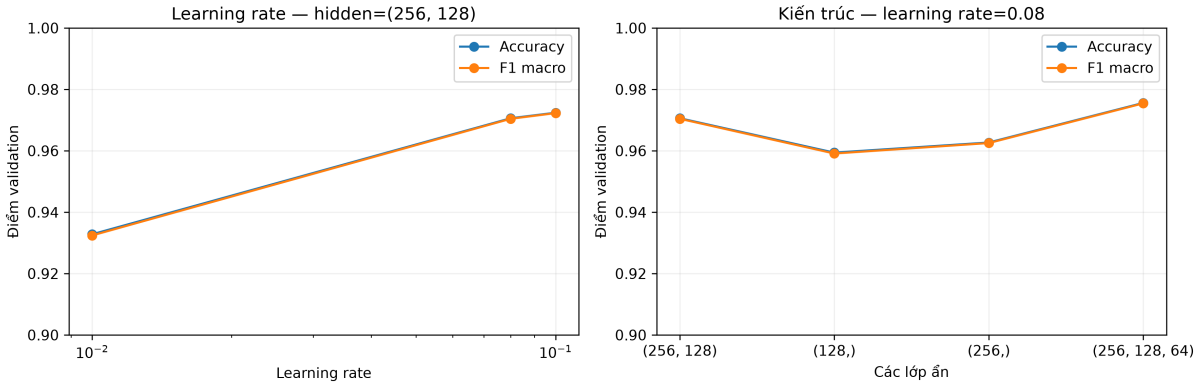
5.4 MLP: lựa chọn kiến trúc trong ngân sách 20 epoch

Bảng 7. Sáu cấu hình MLP NumPy được khảo sát, L2 bằng 0, batch 256.

Lớp ẩn	η	Số tham số	Acc. Validation (%)	F1 Validation (%)
(256, 128)	0,08	235 146	97,060	97,042
(256, 128)	0,01	235 146	93,280	93,242
(256, 128)	0,1	235 146	97,240	97,227
(128,)	0,08	101 770	95,940	95,910
(256,)	0,08	203 530	96,270	96,251
(256, 128, 64)	0,08	242 762	97,560	97,545

Trong nhóm learning rate, 0,1 đạt F1 Validation 97,227%, cao hơn baseline 0,08. Trong nhóm kiến trúc tại 0,08, mạng (256,128,64) đạt F1 97,545%, cao nhất trong sáu cấu hình. Mạng này được chọn và cho accuracy Test 97,63%; code dùng trạng thái epoch 20. Chưa thử mạng ba lớp ẩn với learning rate 0,1, nên không gọi cấu hình được chọn là tối ưu toàn cục.

Accuracy Train 99,494% cao hơn Validation 1,934 điểm phần trăm. Khoảng cách này cho thấy khả



Hình 7. Khảo sát learning rate với mạng (256,128) và khảo sát kiến trúc tại learning rate 0,08.

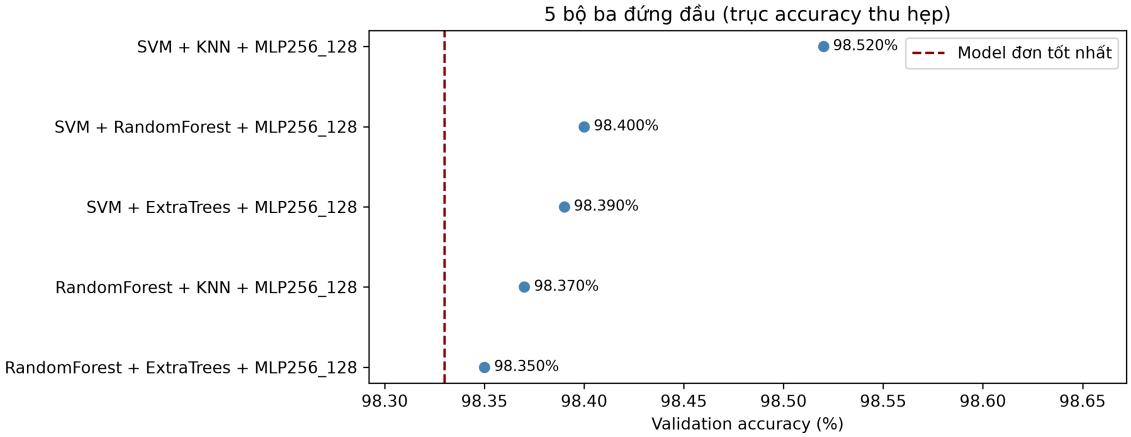
năng khớp Train mạnh hơn dữ liệu chọn cấu hình, nhưng không đủ để khẳng định thêm L2 hoặc early stopping chắc chắn cải thiện Test. Kiểm tra độc lập trên mạng nhỏ cho sai khác gradient W, b khoảng $1,95 \times 10^{-11}$ và $1,92 \times 10^{-11}$. Kết quả này hỗ trợ tính đúng của phép truyền sai số trong mã hiện tại; không giải quyết được khác biệt phiên bản giữa source và mô hình MNIST đã lưu.

5.5 Căn cứ chọn bộ ba mô hình nền

Bảng 8. Sáu ứng viên sau tuning và refit, đánh giá trên Validation bằng argmax xác suất.

Mô hình	Tham số chọn	Acc. (%)	F1 (%)	Số lỗi
SVM	'C': 10	98,330	98,321	167
MLP (256,128)	'alpha': 0.01	98,190	98,180	181
KNN	'n_neighbors': 3	97,200	97,196	280
ExtraTrees	'min_samples_leaf': 1	97,090	97,076	291
RandomForest	'min_samples_leaf': 1	96,690	96,676	331
Softmax_sklearn	'C': 0.1	92,140	92,053	786

SVM cao nhất với accuracy 98,33%, được chốt làm mô hình đơn tham chiếu trên Validation. Nếu dùng `SVC.predict`, accuracy Validation là 98,26%; báo cáo dùng nhất quán argmax xác suất để khớp quy tắc voting và stacking.



Hình 8. Các bộ ba dẫn đầu theo accuracy soft voting trên Validation. Trục hoành chỉ hiển thị một khoảng hẹp để thấy chênh lệch, không bắt đầu tại 0.

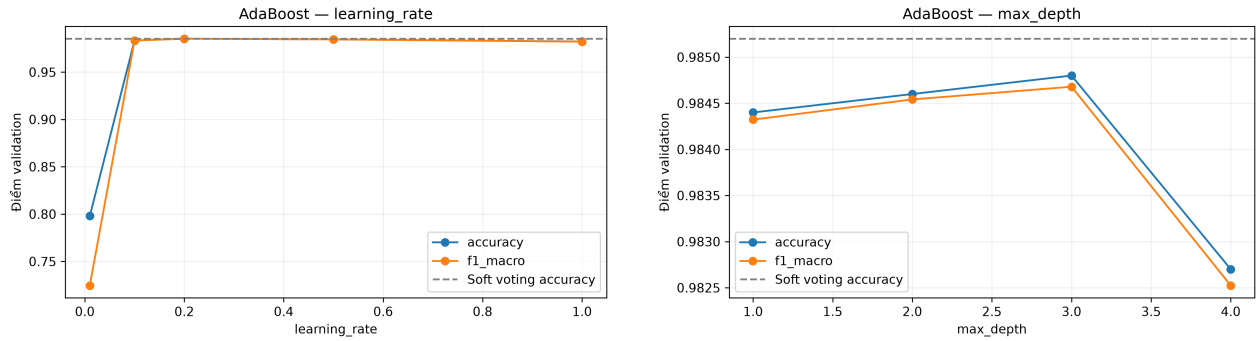
SVM–KNN–MLP (256,128) đạt 98,52%, cao hơn cặp tốt nhất SVM–MLP (98,38%) và SVM đơn (98,33%). Bộ ba cùng sai 81 ảnh, có oracle 99,19% và mean double-fault 1,06%. Oracle không phải chất lượng thực tế của voting. Trên cùng Validation đã dùng chọn nhóm, chênh lệch với các nhóm SVM–Random Forest–MLP (98,40%) và SVM–Extra Trees–MLP (98,39%) còn nhỏ; không khẳng định nhóm dẫn đầu luôn tốt nhất khi đổi mẫu hoặc seed.

Bảng 9. Ablation trên Validation: thêm lại một thành viên vào cặp còn lại.

Thành viên thêm lại	Acc. cặp (%)	Acc. bộ ba (%)	Sửa đúng	Tạo lỗi	Tăng (đpt)
SVM	98,100	98,520	56	14	0,42
KNN	98,380	98,520	32	18	0,14
MLP (256,128)	98,060	98,520	71	25	0,46

KNN có accuracy đơn thấp hơn SVM và MLP, nhưng thêm KNN vào cặp này sửa 32 lỗi và tạo 18 lỗi, tăng ròng 14 ảnh. Đây là bằng chứng trực tiếp cho đóng góp của KNN trong tổ hợp đã chọn. Không thể thay bằng suy luận rằng thành viên yếu hơn luôn giúp tăng đa dạng có ích.

5.6 Stacking, soft voting và trọng số voting



(a) Thay learning rate; depth 2, 100 cây. **(b)** Thay depth; learning rate 0,5, 100 cây.

Hình 9. Khảo sát tầng meta trên OOF; chấm điểm bằng Validation. Đường đứt là accuracy soft voting.

AdaBoost chọn $\eta_A = 0,2$, depth 2, 100 cây, đạt accuracy/F1 Validation 98,530%/98,524%. Learning rate 0,01 chỉ đạt accuracy 79,80% trong cùng 100 cây. Tại learning rate 0,5, tăng số cây từ 100 lên 300 làm accuracy Validation giảm từ 98,46% xuống 98,28%. Tăng độ phức tạp không bảo đảm tốt hơn trên tập chọn cấu hình.

Weighted voting thử bảy bộ trọng số; bộ đều $(1/3, 1/3, 1/3)$ được chọn. Do đó weighted soft voting trùng soft voting trong lượt chạy, không phải một cải thiện mới. Hai bộ trọng số ưu tiên KNN cho accuracy Validation 98,17% và 98,12%, thấp hơn 98,52% của trọng số đều. Chi tiết khảo sát được giữ trong CSV kèm mã nguồn báo cáo.

Stacking sửa 40 lỗi của SVM nhưng tạo 19 lỗi, tăng ròng 21 ảnh, tương ứng 0,21 điểm phần trăm. So với soft voting, chỉ tăng ròng hai ảnh (0,02 điểm). Kiểm định cặp cho $p = 0,8714$ khi so voting, nên chưa có bằng chứng về khác biệt accuracy trên Test này ở ngưỡng 0,05. So SVM, $p = 0,0086$ cho thấy chênh lệch cặp của lượt chạy rõ hơn; đây không phải kết luận về độ ổn định của thuật toán khi huấn luyện lại hoặc sau hiệu chỉnh nhiều so sánh.

AdaBoost có thể học quy tắc chọn lớp từ tương quan các xác suất mà trung bình đều không biểu diễn. Tuy nhiên, số lỗi tạo thêm gần bằng số lỗi sửa được khi so voting. Những khả năng như giới

Bảng 10. Mô hình nền và các phương án gộp trên Test; P, R, F1 là macro.

Mô hình	Acc. (%)	P (%)	R (%)	F1 (%)	Lỗi
SVM	98,330	98,333	98,318	98,325	167
KNN	97,000	97,024	96,962	96,983	300
MLP (256,128)	98,260	98,259	98,247	98,252	174
Stacking AdaBoost	98,540	98,540	98,534	98,536	146
Soft voting	98,520	98,524	98,510	98,516	148
Weighted soft voting	98,520	98,524	98,510	98,516	148

Bảng 11. Stacking so với hai mốc đã chốt: số ảnh Test đổi trạng thái đúng/sai và kiểm định cặp bổ sung.

Tham chiếu	Sửa đúng (b)	Tạo lỗi (c)	Tăng ròng	p hai phía
SVM	40	19	21	0,0086
Soft voting	20	18	2	0,8714

hạn cây nông, sai lệch giữa xác suất OOF và xác suất refit hay mức hiệu chuẩn là giả thuyết cần thí nghiệm riêng, chưa được chứng minh bởi bảng độ đo hiện có.

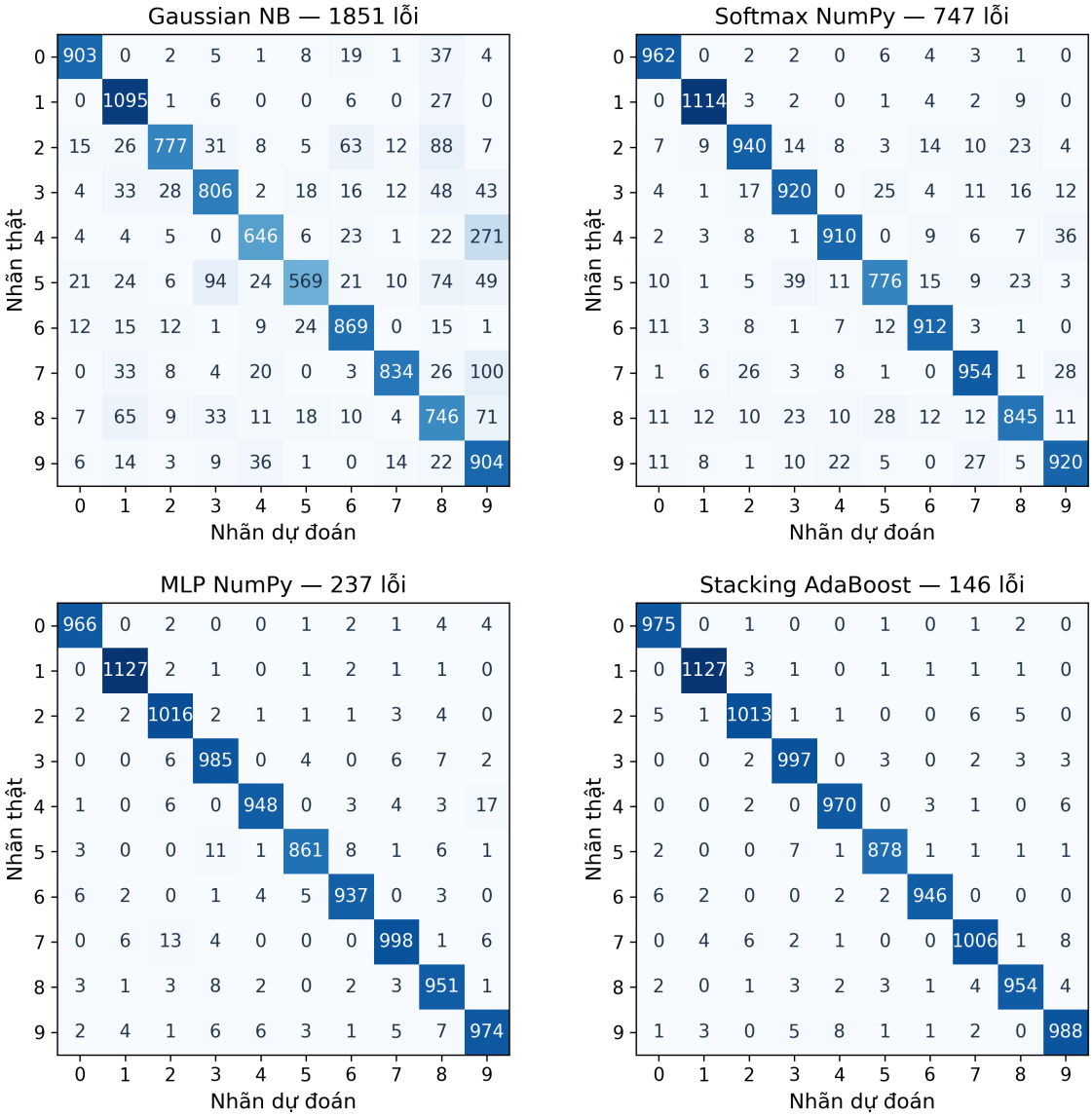
5.7 Confusion matrix và lỗi theo chữ số

Gaussian nhầm $4 \to 9$ tới 271 ảnh; recall thấp nhất ở lớp 5 là 63,789%. Softmax nhầm nhiều nhất $5 \to 3$ (39 ảnh), tiếp theo $4 \to 9$ (36 ảnh); hai hướng $7 \to 9$ và $8 \to 5$ cùng 28 ảnh. Recall lớp 8 của Softmax chỉ 86,756%, thấp nhất dù accuracy chung trên 92%.

Bảng 12. Recall Test theo chữ số. Mỗi cột mô hình tính trên cùng số ảnh thật của lớp.

Lớp	Số ảnh Test	Gaussian (%)	Softmax (%)	MLP (%)	Stacking (%)
0	980	92,14	98,16	98,57	99,49
1	1135	96,48	98,15	99,30	99,30
2	1032	75,29	91,09	98,45	98,16
3	1010	79,80	91,09	97,52	98,71
4	982	65,78	92,67	96,54	98,78
5	892	63,79	87,00	96,52	98,43
6	958	90,71	95,20	97,81	98,75
7	1028	81,13	92,80	97,08	97,86
8	974	76,59	86,76	97,64	97,95
9	1009	89,59	91,18	96,53	97,92

MLP giảm tổng lỗi còn 237, nhưng vẫn nhầm $4 \to 9$ 17 ảnh và $7 \to 2$ 13 ảnh. Stacking còn 146 lỗi; các hướng lớn nhất gồm $7 \to 9$ và $9 \to 4$ cùng 8 ảnh. Số đếm cho thấy lỗi còn tập trung ở một số cặp lớp; chưa đủ để quy nguyên nhân cụ thể cho nét nghiêng, nét thiếu hoặc nhiễu khi chưa phân loại và kiểm tra từng ảnh lỗi.



Hình 10. Confusion matrix Test của bốn phương pháp; hàng là nhãn thật, cột là dự đoán. Hình được dựng từ CSV gốc, giữ nguyên số đếm trong mọi ô.

6 Mở rộng: CNN, MLP và tăng cường dữ liệu

6.1 Thiết kế so sánh có kiểm soát

Phần mở rộng dùng riêng bộ kết quả `cnn_mlp_augmentation`: hai kiến trúc, mỗi kiến trúc chạy ở hai chế độ `none` (không augmentation) và `all` (tổ hợp xoay, dịch, nhiễu). MLP260 là mạng 784–260–10 với ReLU, có 206 710 tham số. CNN dùng hai lớp tích chập 3×3, padding 1, lần lượt 16 và 32 kênh; mỗi lớp theo sau bởi ReLU và max pooling 2×2. Đầu ra được flatten và đưa qua FC128–ReLU–FC10. Số tham số CNN là 206 922, chỉ nhiều hơn MLP260 212 tham số.

Một phần tử đầu ra tích chập có dạng

$$Z_{k,u,v} = b_k + \sum_{c=1}^{C_{\text{in}}} \sum_{a=0}^{2} \sum_{b=0}^{2} W_{k,c,a,b}\, A_{c,u+a-1,v+b-1}, \qquad H = \max(0, Z), \tag{24}$$

Bảng 13. Kích thước tensor và số tham số CNN theo đúng các lớp trong notebook.

Lớp	Đầu ra (bỏ chiều batch)	Số tham số
Đầu vào	$1 \times 28 \times 28$	0
Conv 3×3, $1 \to 16$; ReLU	$16 \times 28 \times 28$	$(3 \cdot 3 \cdot 1 + 1)16 = 160$
MaxPool 2×2	$16 \times 14 \times 14$	0
Conv 3×3, $16 \to 32$; ReLU	$32 \times 14 \times 14$	$(3 \cdot 3 \cdot 16 + 1)32 = 4\,640$
MaxPool 2×2; flatten	$32 \times 7 \times 7 \to 1568$	0
Linear $1568 \to 128$; ReLU	128	$1568 \cdot 128 + 128 = 200\,832$
Linear $128 \to 10$	10 logits	$128 \cdot 10 + 10 = 1\,290$
Tổng		206 922

với phần ngoài ảnh được padding bằng 0. Max pooling lấy giá trị lớn nhất trong ô 2×2. Các lớp cuối trả logits; `F.cross_entropy` xử lý logits trực tiếp, không đặt softmax trước loss. Hai mạng dùng SGD momentum 0,9, learning rate 0,01, batch 256, 15 epoch, seed 42; chọn checkpoint có CE Validation sạch thấp nhất, ưu tiên epoch sớm khi bằng nhau.

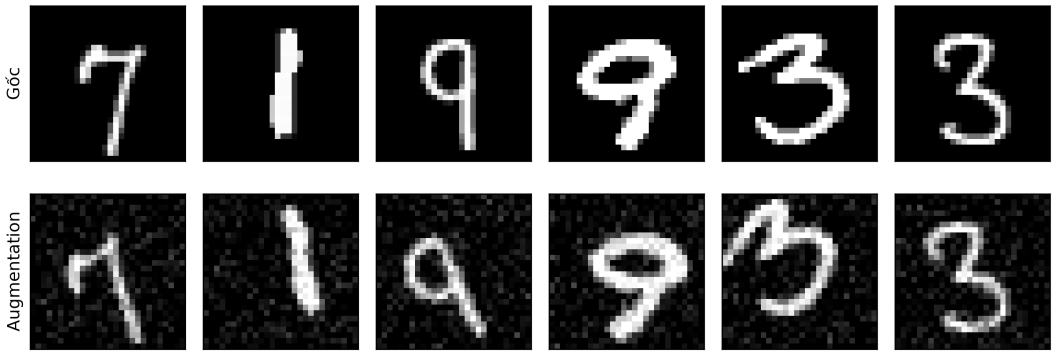
Trong mỗi kiến trúc, hai chế độ dùng cùng khởi tạo. DataLoader có cùng seed cho shuffle; augmentation có bộ sinh ngẫu nhiên riêng và cùng chuỗi biến đổi giữa hai kiến trúc. Số tham số gần nhau giúp giảm một yếu tố gây nhiễu, nhưng không làm hai mạng bằng nhau về số phép tính, độ khó tối ưu hoặc năng lực biểu diễn.

6.2 Phép tăng cường dữ liệu và cách đo độ bền

Augmentation chỉ áp dụng trong các mini-batch Train:

$$\theta \sim \mathcal{U}(-30^\circ, 30^\circ), \quad \Delta x, \Delta y \sim \mathcal{U}(-4, 4), \quad \tilde{I} = \mathrm{clip}_{[0,1]}\left(T_{\theta,\Delta x,\Delta y}(I) + \epsilon\right), \quad \epsilon \sim \mathcal{N}(0, (0{,}1)^2). \tag{25}$$

Trong biểu thức cuối, $0{,}1$ là độ lệch chuẩn nhiễu (dùng dấu phẩy thập phân). Code dùng `affine_grid` và `grid_sample`, nội suy bilinear, `padding_mode='zeros'`, `align_corners=False`. Train và Validation được đánh giá trên ảnh sạch ở cuối mỗi epoch, kể cả khi huấn luyện có augmentation.



Hình 11. Các ảnh Train gốc và tổ hợp augmentation dùng trong notebook. Hình minh họa không thay thế thí nghiệm tách riêng từng phép biến đổi.

Độ bền được đánh giá trên các bản sao Test sau khi huấn luyện: dịch 0–4 pixel theo một trong bốn hướng và xoay với độ lớn 0, 10, 20, 30 độ. Hướng dịch và dấu góc xoay được cố định theo ảnh với

seed riêng, dùng cùng ảnh biến đổi cho mọi mô hình. Không thêm nhiễu trong phép đánh giá độ bền này, không tổng hợp nhiều dự đoán Test-time augmentation, và không huấn luyện lại trên Test.

6.3 Chất lượng trên ảnh sạch

Bảng 14. Bốn trường hợp trong cùng ngân sách 15 epoch. Acc./F1 theo %; thời gian fit theo giây.

Mạng	Chế độ	Train	Val.	Test	F1 Test	Epoch chọn	Fit
MLP260	none	96,430	95,750	95,980	95,943	15	18,307
MLP260	all	87,400	87,800	87,630	87,328	12	34,858
CNN_FC128	none	99,120	98,380	98,570	98,560	13	120,845
CNN_FC128	all	98,102	98,100	98,240	98,233	14	110,385

CNN không augmentation đạt accuracy 98,57%, cao hơn MLP260 không augmentation 2,59 điểm, tương ứng ít hơn 259 lỗi (143 so với 402). Đây là kết quả của hai mạng với số tham số gần bằng nhau trong giao thức đã chọn, phù hợp với lợi ích của tích chập trong khai thác cấu trúc ảnh. Không diễn giải nó là bằng chứng rằng mọi CNN đều tốt hơn mọi MLP.

Bảng 15. Tác động của augmentation trên Test sạch: chế độ all so với none trong cùng kiến trúc.

Mạng	Chênh accuracy (đpt)	Lỗi được sửa	Lỗi tạo thêm
MLP260	-8,35	142	977
CNN_FC128	-0,33	75	108

Augmentation làm accuracy sạch của CNN giảm 0,33 điểm; số lỗi tăng từ 143 lên 176. MLP260 giảm mạnh 8,35 điểm, số lỗi tăng từ 402 lên 1 237. Không thể từ kết quả này kết luận augmentation nói chung có hại: thí nghiệm đang dùng một tổ hợp biến đổi khá mạnh và chỉ 15 epoch, chưa khảo sát cường độ nhẹ hơn hoặc ngân sách dài hơn.

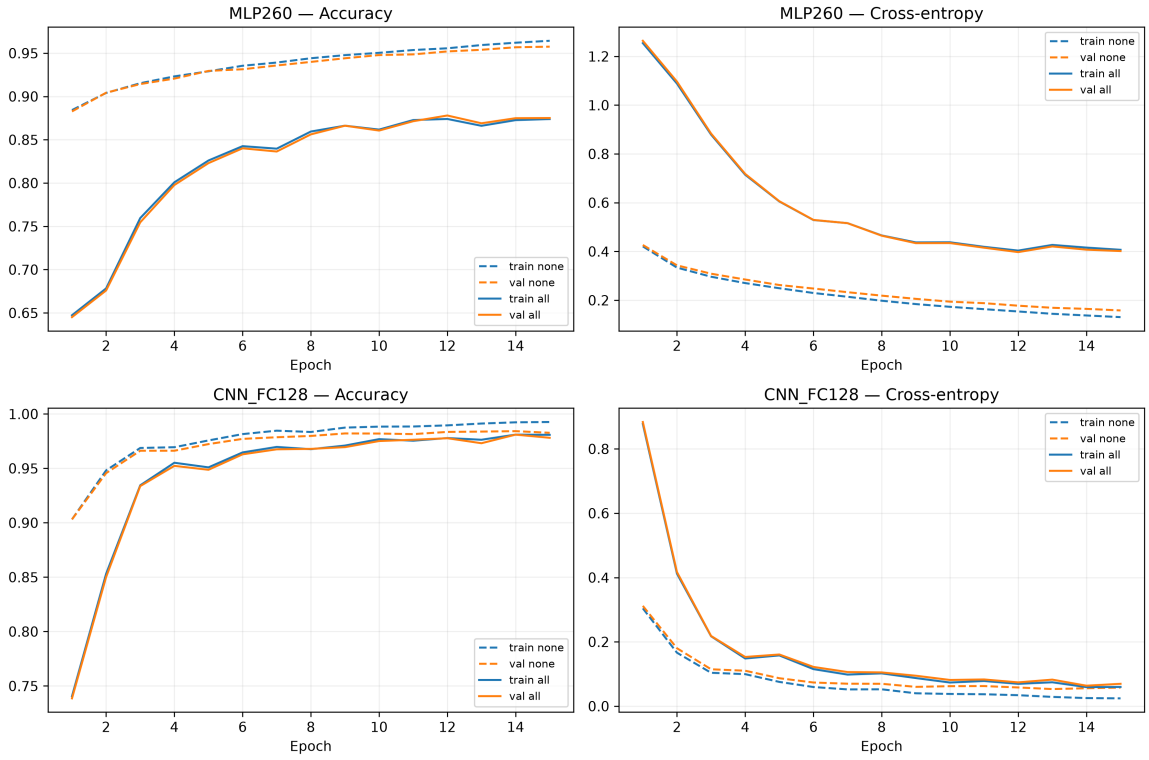
Khoảng cách accuracy Train–Validation của CNN giảm từ 0,740 xuống 0,002 điểm, nhưng Validation đồng thời giảm từ 98,38% xuống 98,10%. Với MLP augmentation, Train chỉ đạt 87,40% và Validation 87,80%. Gap nhỏ không tự chứng minh tổng quát hóa tốt hơn; kết quả cũng phù hợp với khả năng bài toán huấn luyện khó hơn trong ngân sách cố định. Vì chế độ all gộp ba phép biến đổi, chưa xác định được thành phần nào gây ra mức giảm.

6.4 Chất lượng khi ảnh bị dịch và xoay

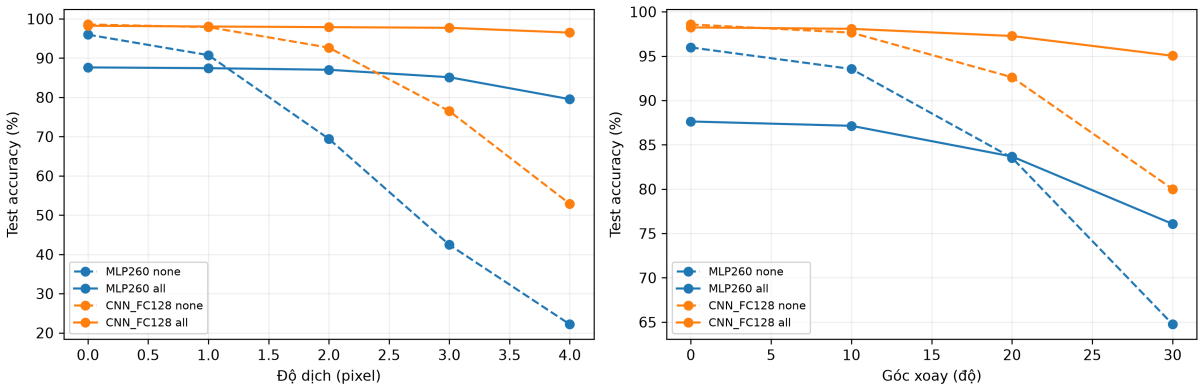
Bảng 16. Một số mốc độ bền; mọi giá trị là accuracy Test (%).

Điều kiện	MLP none	MLP all	CNN none	CNN all
Ảnh sạch	95,98	87,63	98,57	98,24
Dịch 2 pixel	69,43	87,03	92,67	97,89
Dịch 4 pixel	22,18	79,54	52,88	96,50
Xoay 20°	83,53	83,68	92,62	97,27
Xoay 30°	64,75	76,07	80,00	95,04

Ở mức dịch 4 pixel, CNN tăng từ 52,88% lên 96,50% nhờ augmentation, tăng 43,62 điểm; MLP tăng từ 22,18% lên 79,54%. Khi xoay 30 độ, CNN tăng từ 80,00% lên 95,04%, MLP tăng từ 64,75% lên



Hình 12. Đường học của MLP260 và CNN, có/không augmentation. Các đường Train/Validation đều được đo trên ảnh sạch; mỗi điểm dùng tham số cuối epoch.



Hình 13. Accuracy trên các bản sao Test bị dịch hoặc xoay. Đường liền là all, đường đứt là none; không bổ sung nhiễu ở phép kiểm tra này.

76,07%. Augmentation trong lượt chạy đánh đổi độ chính xác sạch lấy độ bền đối với một số phép biến đổi tương ứng với quá trình huấn luyện.

Các kết quả chưa bao phủ nhiễu riêng, nét viết từ camera, ảnh tự vẽ hoặc mọi dạng phân phối mới. CNN có accuracy sạch 98,57%, gần stacking 98,54%, nhưng hai pipeline không có cùng ngân sách tuning và huấn luyện. Chênh lệch ba ảnh không đủ để xếp hạng ổn định hai họ phương pháp.

7 Thảo luận và kết luận

7.1 Đánh đổi độ chính xác, chi phí và khả năng giải thích

Bảng 17. Thời gian fit và dự đoán trên 10 000 ảnh Test, trích từ metrics của từng phương pháp.

Phương pháp	Fit (giây)	Predict (giây)
Gaussian NB	0,338	1,466
Softmax NumPy	11,151	0,009
MLP NumPy	71,609	0,069
SVM	942,403	47,086
KNN	0,034	3,454
MLP (256,128)	474,684	0,087
Soft voting	1417,121	50,627
Stacking AdaBoost	6513,241	50,715

Gaussian NB fit nhanh vì chỉ ước lượng thống kê theo lớp; suy luận phải tính điểm Gaussian cho từng pixel và lớp. Softmax dùng một phép nhân ma trận cho suy luận, cho thời gian thấp trong lượt đo. MLP bổ sung các phép biến đổi phi tuyến và nhiều tham số, đạt chất lượng cao hơn với chi phí huấn luyện tăng. KNN có thời gian fit gần 0 vì chủ yếu lưu dữ liệu; chi phí so khoảng cách chuyển sang giai đoạn dự đoán.

Stacking có thời gian fit ghi nhận 6 513,241 giây, khoảng 1,81 giờ, so với 1 417,121 giây của voting dùng cùng ba mô hình nền, tỷ lệ khoảng 4,60 lần. Thời gian dự đoán Test tương ứng 50,715 và 50,627 giây. Phần lớn chi phí suy luận đến từ việc tạo xác suất nền, đặc biệt SVM; tầng meta chỉ bổ sung một phần nhỏ. Đổi lấy hai ảnh đúng ròng trên Test, stacking chưa cho lợi ích rõ ràng so với voting nếu mục tiêu là đơn giản hóa hệ thống và giảm chi phí huấn luyện.

Bảng 18. Ưu điểm và giới hạn theo cơ chế mô hình và kết quả đã quan sát

Phương pháp	Ưu điểm	Giới hạn
Gaussian NB	Thống kê tham số rõ; fit nhanh; dễ đối chiếu từng prior, mean, variance.	Độc lập có điều kiện và Gaussian chưa phù hợp hoàn toàn với pixel; accuracy thấp nhất.
Softmax	Công thức, gradient gọn; bản đồ trọng số giải thích đóng góp vào logit; suy luận nhanh.	Ranh giới tuyến tính theo pixel, còn 747 lỗi Test.
MLP	Học quan hệ phi tuyến; giảm lỗi rõ so Softmax.	Nhiều tham số; phụ thuộc tối ưu và khởi tạo; khó giải thích từng dự đoán hơn.
Stacking	Kết hợp thông tin xác suất; OOF giảm rò rỉ do dùng dự đoán in-sample.	Tốn nhiều lần fit; lợi ích nhỏ so voting; nhiều thành phần và quy tắc lựa chọn.
CNN	Khai thác cấu trúc không gian; tốt hơn MLP260 trong so sánh gần bằng số tham số.	Độ bền không tự có với dịch/xoay lớn; augmentation có đánh đổi trên ảnh sạch.

Các phép đo thời gian chưa phải benchmark kiểm soát phần cứng, số luồng và tải máy. MLP (128,)

trong khảo sát thậm chí có thời gian ghi nhận cao hơn mạng sâu hơn; không suy ra ít tham số làm chậm hơn về bản chất. Tương tự, CNN augmentation được ghi nhận nhanh hơn baseline khoảng 10,46 giây, không chứng minh augmentation làm tăng tốc. Không cộng thời gian fit của cùng một mô hình qua ba split hoặc cộng trùng cấu hình đã tái sử dụng.

7.2 Giới hạn của kết luận và tính tái lập

Thứ nhất, chỉ một seed được báo cáo. Kiểm định cặp trên Test không thay thế nhiều lượt huấn luyện để đo độ biến động. Validation được dùng nhiều lần cho chọn mô hình và meta, nên mức cải thiện Validation có thể chứa thiên lệch lựa chọn. OOF không biến quy trình này thành nested CV.

Thứ hai, so cùng epoch hoặc gần bằng số tham số chỉ kiểm soát một phần điều kiện thí nghiệm. MLP sklearn có cảnh báo chưa hội tụ; MLP trong ensemble dùng Adam và 60 epoch, còn MLP NumPy dùng SGD và 20 epoch. Không dùng chênh lệch giữa chúng để kết luận về ưu thế của thư viện hoặc cài tay. Phần augmentation chưa có ablation riêng xoay, dịch, nhiễu.

Thứ ba, artifact số liệu nhất quán không đồng nghĩa mã nguồn hiện tại tái lập được số liệu. Các lỗi unpack dữ liệu, khác biệt dtype MLP và các tệp thời gian không đồng bộ đã được ghi rõ ở Phụ lục A. Báo cáo giữ nguyên kết quả, công thức mô tả đúng code hiện có và không khẳng định toàn bộ notebook đã được chạy lại thành công. Ngoài ra, phần lõi AdaBoost dùng thư viện cần được đối chiếu với yêu cầu tự cài đặt của đề; không thể giải quyết thiếu sót thực hiện bằng cách đổi tên phương pháp trong báo cáo.

7.3 Kết luận và hướng phát triển

Trên bộ kết quả đã lưu, accuracy Test của Gaussian NB, Softmax, MLP NumPy và Stacking AdaBoost lần lượt là 81,49%, 92,53%, 97,63% và 98,54%. Phân tích theo lớp cho thấy accuracy chung cần đi kèm recall và confusion matrix để nhận diện những chữ số còn bị bỏ sót. Với ensemble, số lỗi được sửa và số lỗi phát sinh giải thích trực tiếp vì sao tầng meta chỉ hơn soft voting hai ảnh đúng ròng.

Trong phần mở rộng, CNN có gần bằng số tham số với MLP260 nhưng tốt hơn trên ảnh sạch ở ngân sách 15 epoch. Augmentation mạnh làm giảm accuracy sạch, đồng thời cải thiện độ bền xoay/dịch; không có một cấu hình tốt nhất cho mọi mục tiêu chất lượng và chi phí.

Ưu tiên tiếp theo là đồng bộ source, notebook và metadata; lưu split và phiên bản mã; hoàn thiện lõi ensemble, rồi đánh giá nhiều seed và ablation augmentation trên Validation. Nếu dùng Test đã xem để chỉnh cấu hình, cần tập kiểm thử độc lập mới.

Phân công công việc và khai báo hỗ trợ

Thông tin thành viên theo bìa gốc; nhiệm vụ và tỷ lệ phải được nhóm xác nhận

Họ và tên	MSSV	Nhiệm vụ, sản phẩm phụ trách	Tỷ lệ
Trần Hùng Luyện	24110201	[Nhóm điền công việc thực tế]	[Điền] %
Lê Mỹ Tài	24110235	[Nhóm điền công việc thực tế]	[Điền] %

Tỷ lệ đóng góp phải được nhóm xác nhận, tổng bằng 100%.

Khai báo sử dụng AI. ChatGPT hỗ trợ rà soát mã nguồn, đối chiếu số liệu, diễn đạt công thức, mã giả và biên tập LaTeX. AI không tạo số liệu huấn luyện. Kiểm định và kiểm tra số học bổ sung được ghi riêng. Nhóm chịu trách nhiệm về mã, kết quả và sản phẩm nộp [1].

Tài liệu tham khảo

[1] Khoa Toán – Tin học, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM. *Đồ án thực hành – môn Nhận dạng*, Đồ án 1: Nhận dạng chữ số viết tay bằng các phương pháp phân lớp. Tài liệu đề bài được cung cấp cùng dự án.

[2] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. Chương 2, mục 4.3.4 và chương 5. Bản tài liệu do người dùng cung cấp.

[3] Trương Phước Hưng. *Ensemble Methods*, Bài giảng 06, môn Nhận dạng mẫu, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM. Các phần Voting, Boosting và Stacking. Bản bài giảng được cung cấp cùng dự án.

[4] scikit-learn developers. *GaussianNB: API reference*. Tài liệu trực tuyến; tham số `var_smoothing`, thuộc tính `epsilon_`. https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.GaussianNB.html. Truy cập ngày 08/11/2026.

[5] scikit-learn developers. *Linear Models: Logistic Regression, Multinomial Case*. Hàm mục tiêu và quy ước regularization. https://scikit-learn.org/stable/modules/linear_model.html#multinomial-case. Truy cập ngày 08/11/2026.

[6] scikit-learn developers. *AdaBoostClassifier: API reference*. Thuật toán AdaBoost đa lớp; tài liệu tham chiếu tới J. Zhu, H. Zou, S. Rosset và T. Hastie, "Multi-class AdaBoost", *Statistics and Its Interface*, 2(3), 349–360, 2009. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html. Truy cập ngày 08/11/2026.

[7] scikit-learn developers. *Ensembles: Stacked Generalization*. Quy trình học tầng cuối từ dự đoán cross-validation. https://scikit-learn.org/stable/modules/ensemble.html#stacked-generalization. Truy cập ngày 08/11/2026.

A Đối chiếu mã nguồn, số liệu và khả năng tái lập

A.1 Các khác biệt cần xử lý trước lần chạy tiếp theo

Những phát hiện dưới đây được ghi từ tệp đính kèm. Chúng không được sửa ngầm rồi gán kết quả cũ cho một lượt huấn luyện mới. Toàn bộ số liệu trong phần chính vẫn là kết quả đã lưu.

Bảng 19. Các điểm chưa đồng bộ giữa mã nguồn và artifact, cùng cách xử lý cụ thể

Vị trí	Bằng chứng	Cách xử lý
Loader và Softmax	`load_prepared_mnist()` trả 4 mảng, nhưng notebook 03 nhận 6 biến.	Nạp bốn mảng rồi chia 60 000 ảnh thành Train/Validation trong notebook theo cùng seed và stratify.
EDA và Gaussian	Notebook 01/02 gán kết quả split theo thứ tự $X_{\text{train}}, y_{\text{train}}, X_{\text{val}}, y_{\text{val}}$, khác thứ tự trả về của sklearn. Output lưu lại có kích thước đúng.	Đổi thành $X_{\text{train}}, X_{\text{val}}, y_{\text{train}}, y_{\text{val}}$. Output cũ không chứng minh dòng code hiện tại chạy đúng.
MLP: dtype	Notebook/config ghi float64; source hiện tại ép đầu vào, trọng số và bias sang float32. Joblib đã lưu chứa cả trọng số và bias float64.	Đồng bộ phiên bản source dùng huấn luyện, dtype và metadata; chưa khẳng định lượt chạy cũ được tạo bởi đúng source hiện tại.
MLP: khởi tạo	Source dùng uniform theo $\sqrt{6/(d_{\text{in}} + d_{\text{out}})}$ cho cả W, b; bản thảo cũ mô tả He normal và bias 0.	Công thức báo cáo đã sửa theo source. Không suy đoán khởi tạo lượt chạy từ dtype hay trọng số cuối.
Softmax: thời gian	`metrics.csv` ghi fit 11,150642 s; `comparison.csv` ghi 11,923309 s; `validation_comparison.csv` ghi 14,766817 s cho cùng tên mô hình.	Dùng `metrics.csv` nhất quán. Không trộn các mốc đo khác nhau để tính speedup.
OOF	Markdown đầu notebook 06 nói 10 fold; biến cấu hình, config và fold_id đều là 5 fold.	Mô tả theo mã và artifact: 5 fold, mỗi fold holdout 10 000 ảnh.
CNN trên GPU	torch.cuda.manual_seed_all(seed) dùng tên `seed` chưa định nghĩa trong notebook 09; nhánh chỉ chạy khi có CUDA.	Đổi `seed` thành `SEED` trước khi chạy GPU. Bộ kết quả báo cáo ghi CPU nên không đi qua nhánh này.
Lõi ensemble	Các estimator và AdaBoost được gọi từ scikit-learn; đề yêu cầu tự cài phần tính toán lõi.	Cần tự cài, chạy và đối chiếu phần ensemble theo yêu cầu, hoặc có xác nhận phạm vi cho phép từ giảng viên. Không thay số liệu thư viện bằng số liệu chưa chạy.

Phần sửa dữ liệu tối thiểu sau đây phù hợp loader bốn giá trị và dùng được cho EDA, Gaussian hoặc Softmax. Đây là hướng sửa, không phải tuyên bố các notebook đã được chạy lại:

```
X, y, X_test, y_test = load_prepared_mnist()
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=10000, stratify=y, random_state=42)
```

A.2 Bản đồ công thức và cài đặt

Bảng 20. Các thành phần cài đặt tương ứng với công thức trong phần chính

Thành phần	Phương thức / thao tác trong mã	Công thức
Gaussian	`fit`: prior, mean, var; `_log_joint`: log-score	(2)–(3)
Softmax	`softmax`, `_grad`, `_loss`; checkpoint trong `fit`	(5)–(9)
MLP NumPy	`_init_para`, `_forward`, `_backward`, `_loss`	(10)–(14)
Voting / OOF	Mean xác suất; clone estimator ngoài fold và ghép cột	(17)–(18)
AdaBoost	`make_meta`, estimator thư viện trên 30 đặc trưng	(19)–(22)
CNN / augmentation	`make_cnn`, `apply_affine`, `augment_batch`	(24)–(25)

A.3 Nguồn số liệu và kiểm tra đã thực hiện

Các bảng trong `tables/` được tạo từ CSV riêng của từng phương pháp, không dùng results/comparison/ làm nguồn chính. Thư mục `evidence/` giữ các CSV/JSON và dự đoán cần cho đối chiếu. `source_manifest.json` lưu tên, dung lượng và SHA-256 của các tệp nguồn đã đọc; `code_snapshot/` giữ nguyên source NumPy. Các tệp này phục vụ truy xuất nguồn gốc, không chứa dữ liệu MNIST gốc hay các trọng số lớn.

Kiểm tra số liệu tính lại accuracy, P/R/F1 macro từ confusion matrix hoặc dự đoán; kiểm tra tổng số mẫu, thứ tự nhãn và số lỗi. Các vector nhãn Train/Validation của Gaussian, Softmax và stacking trùng nhau; nhãn Test của Gaussian, Softmax và CNN cũng trùng nhau. Fold ID của OOF có năm giá trị 0–4, mỗi giá trị xuất hiện đúng 10 000 lần. Kiểm tra đầy đủ có thể lặp lại bằng `scripts/verify_evidence.py`; kết quả chi tiết được lưu trong `evidence/verification.json`.

Bảng 21. Nguồn chính cho từng nhóm bảng và hình

Nội dung	Tệp / vị trí nguồn
EDA	Output nhúng của `01_eda.ipynb`: phân phối Train, ảnh trung bình, thống kê pixel.
Gaussian	gaussian/metrics.csv, epsilon_survey.csv, predictions.npz, parameter_checks.csv.
Softmax	softmax/metrics.csv, learning_rate.csv, l2.csv, các confusion CSV, gradient check.
MLP	mlp/metrics.csv, survey.csv, các confusion CSV, config và joblib gốc dùng kiểm tra dtype.
Chọn nền	select_base_models/singles.csv, triples.csv, selected_ablation.csv, selection.json.
Stacking	stacking_adaboost/metrics.csv, hyperparameters.csv, error_analysis_test.csv, config, fold ID.
CNN	cnn_mlp_augmentation/metrics.csv, runs.csv, robustness.csv, test_predictions.npz.

B Các độ đo bổ sung trên ba tập

B.1 Mô hình nền và voting trên Train, Validation

Bảng 22. P/R/F1 macro của các phương án ensemble trên Train và Validation (%). Train được đo sau refit, là in-sample.

Mô hình	Tập	Acc.	P	R	F1
SVM	Train	99,990	99,990	99,990	99,990
KNN	Train	100,000	100,000	100,000	100,000
MLP (256,128)	Train	99,990	99,990	99,990	99,990
Stacking AdaBoost	Train	99,998	99,998	99,998	99,998
Soft voting	Train	100,000	100,000	100,000	100,000
Weighted soft voting	Train	100,000	100,000	100,000	100,000
SVM	Validation	98,330	98,323	98,322	98,321
KNN	Validation	97,200	97,255	97,163	97,196
MLP (256,128)	Validation	98,190	98,177	98,187	98,180
Stacking AdaBoost	Validation	98,530	98,523	98,526	98,524
Soft voting	Validation	98,520	98,520	98,512	98,515
Weighted soft voting	Validation	98,520	98,520	98,512	98,515

KNN với trọng số khoảng cách có thể đạt 100% trên chính dữ liệu được lưu làm tập láng giềng. Điểm này không phải bằng chứng về độ chính xác trên dữ liệu mới. Tương tự, stacking Train đạt 99,998% (một ảnh sai) không được dùng thay accuracy OOF hay Test.

B.2 CNN và MLP260: độ đo macro đầy đủ

Bảng 23. Accuracy, precision, recall và F1 macro của phần mở rộng, đơn vị %.

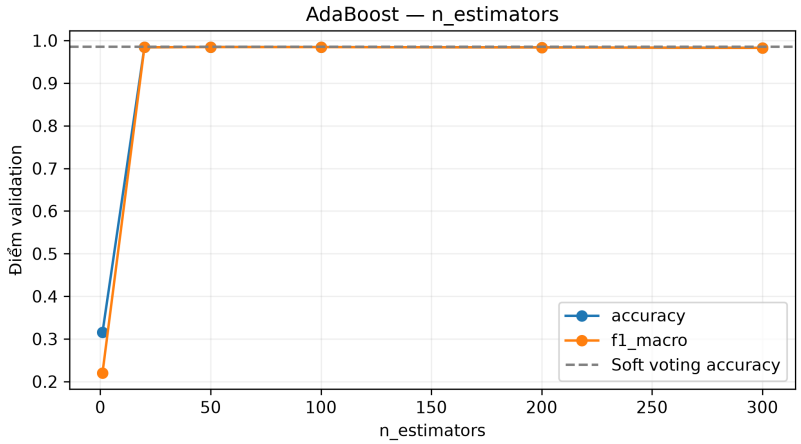
Mạng	Chế độ	Tập	Acc.	P	R	F1
MLP260	none	Train	96,430	96,418	96,394	96,404
MLP260	none	Validation	95,750	95,733	95,717	95,722
MLP260	none	Test	95,980	95,961	95,938	95,943
MLP260	all	Train	87,400	88,655	87,189	87,114
MLP260	all	Validation	87,800	88,954	87,597	87,581
MLP260	all	Test	87,630	88,793	87,459	87,328
CNN_FC128	none	Train	99,120	99,124	99,118	99,119
CNN_FC128	none	Validation	98,380	98,376	98,366	98,369
CNN_FC128	none	Test	98,570	98,572	98,558	98,560
CNN_FC128	all	Train	98,102	98,113	98,078	98,088
CNN_FC128	all	Validation	98,100	98,104	98,073	98,081
CNN_FC128	all	Test	98,240	98,263	98,215	98,233

Các độ đo Train của chế độ all cũng được đánh giá trên ảnh sạch. Không diễn giải các dòng này thành accuracy trên ảnh đã augmentation trong lúc cập nhật gradient.

C Khảo sát chi tiết tầng meta

Bảng 24. Các cấu hình AdaBoost đã thử: fit trên OOF, đánh giá Validation. Các cấu hình trùng baseline chỉ ghi một lần.

Learning rate	Max depth	Số cây	Acc. (%)	F1 (%)
0,01	2	100	79,800	72,447
0,1	2	100	98,340	98,334
0,2	2	100	98,530	98,524
0,5	2	100	98,460	98,454
1	2	100	98,230	98,214
0,5	1	100	98,440	98,432
0,5	3	100	98,480	98,468
0,5	4	100	98,270	98,252
0,5	2	1	31,520	22,023
0,5	2	20	98,410	98,406
0,5	2	50	98,480	98,472
0,5	2	200	98,390	98,382
0,5	2	300	98,280	98,269



Hình 14. Khảo sát số cây ở learning rate 0,5 và depth 2. Đường đứt là accuracy soft voting; các mốc số cây chưa thể hiện mọi tổ hợp learning rate và depth.

Các khảo sát learning rate/depth/số cây dùng những lát cắt khác nhau qua không gian tham số. Chỉ có thể nói cấu hình được chọn tốt nhất trong các cấu hình thực sự đã đánh giá, không suy ra độ sâu 2 hoặc 100 cây là lựa chọn tối ưu cho mọi learning rate.